MediFlow — Database Architecture & Demonstration

PostgreSQL-Based Healthcare Data Management & Single Source of Truth

1. Executive Project Summary

MediFlow is an intelligent clinical appointment booking, live virtual waiting-room, and AI pre-consultation platform designed to eliminate physical hospital queue congestion. The platform provides real-time patient queue tracking, specialist discovery across multiple metropolitan hubs, structured conversational pre-consultation intake, and automated physician discharge summaries.

This demonstration document outlines the persistent PostgreSQL database architecture that serves as the single source of truth for all clinical data across both Web and Mobile client applications.

2. Key Database Objectives

- **Single Source of Truth:**
  One central PostgreSQL database powers both MediFlow Web and MediFlow Mobile without data fragmentation.
- **Backend-Mediated Security:**
  Zero direct database exposure to client devices. All interactions flow through an authenticated Express.js API tier.
- **Clinical Data Integrity:**
  Enforces strict relational foreign keys between users, patient profiles, doctors, appointments, and consultation reports.
- **High Performance Indexing:**
  Optimized B-tree indexes for low-latency live queue sorting, doctor discovery, and authentication lookups.
- **Cloud & Render Ready:**
  Engineered with connection pooling and SSL auto-detection for seamless production deployment on Render.

3. End-to-End System Architecture

The MediFlow platform implements a decoupled, secure 3-tier healthcare architecture. Both the Web and Mobile frontends interact exclusively with the unified Express.js backend.

MediFlow Web Application
React 19 + Vite + Tailwind CSS
Frost UI Glassmorphic Interface

MediFlow Mobile Application
React Native Mobile Client
Patient Waiting Room & Digital Vault

%¼ HTTPS REST & Server-Sent Events (SSE) %¼

Unified Express.js Backend Server (Node.js API Tier)

- JWT Authentication & Role-Based Access Control (Patient & Doctor roles)
- Real-time SSE Live Queue Broadcast Manager & Turn-call Notification Dispatcher
- PostgreSQL Database Connection Pool (node-postgres pg.Pool with SSL)
- HL7 FHIR R4 Interoperability & Synthetic Healthcare Integration Layer

%¼
Managed Connection Pool Queries

ONE Persistent PostgreSQL Database (Single Source of Truth)

- 9 Relational Tables with Foreign Key Constraints
- Stores Users, Patients, Doctors, Appointments, Live Queues, Medical Reports, Vault & Alerts
- Fully Isolated: Direct client access is strictly forbidden by network architecture
- Production Configuration compatible with Render managed cloud database

KEY ARCHITECTURAL HIGHLIGHT:

Web and Mobile clients NEVER connect directly to PostgreSQL. All operations are mediated by backend APIs, guaranteeing zero credential leakage and unified business logic.

4. Detailed Overview of the 9 Database Tables

1. Table: locations
[Regional Hub Cities]

Stores 5 major metropolitan hubs (Chennai, Bangalore, Hyderabad, etc.) and popular medical clusters.

2. Table: users
[User Credentials & Roles]

Stores login identifiers, bcrypt password hashes, and system roles (patient / doctor).

3. Table: patient_profiles
[Patient Clinical Info]

Stores verified patient demographics, blood group, contact details, and emergency medical alerts.

4. Table: doctors
[Specialist Directory]

Stores 70 specialist profiles, consultation fees, ratings, reviews, hospital affiliations, and time slots.

5. Table: appointments
[Live Queue & Bookings]

Stores booking tokens (Q-01), real-time queue states, AI intake responses, and doctor notes.

6. Table: consultation_reports
[Discharge Summaries]

Stores structured physician clinical assessments, diagnostic findings, prescriptions, and follow-ups.

7. Table: notifications
[Real-Time Alert Dispatch]

Stores patient notifications for queue turn alerts, confirmed bookings, and report readiness.

8. Table: family_members
[Dependent Health Records]

Stores family member profiles linked to primary patients for coordinated care management.

9. Table: medical_documents
[Digital Health Vault]

Stores patient document metadata (lab blood panels, ECG scans) and doctor sharing consent flags.

5. Entity Relationship & Data Flow Architecture

The database establishes relational foreign key constraints and logical grouping to ensure strict clinical referential integrity.

users (Primary Key: id)

Type: 1-to-1 / 1-to-N Mapping

'œ patient_profiles (Foreign Key: user_id -> users.id)

Each authenticated user account references an associated patient clinical profile containing demographics and emergency alerts.

doctors (Primary Key: id)

Type: 1-to-Many Relationship

'œ appointments (Foreign Key: doctor_id -> doctors.id)

Each specialist doctor manages multiple patient appointments. Deleting a doctor cascades appropriately to appointments.

appointments (Primary Key: id)

Type: 1-to-1 Clinical Lifecycle

'œ consultation_reports (Foreign Key: appointment_id -> appointments.id)

When a consultation completes, an official structured medical report is generated linking directly to the original appointment token.

patient_profiles (Primary Key: id)

Type: Logical Patient Grouping

'œ notifications, family_members, medical_documents

Patient records logically group notifications, family dependent profiles, and encrypted health document records in the vault.

IDEMPOTENT INSERTION & DEPENDENCY ORDER:

locations 'œ users 'œ patient_profiles 'œ doctors 'œ appointments 'œ consultation_reports 'œ notifications / family / documents

6. Security Architecture & Privacy Safeguards

' Bcrypt Password Hashing

All user credentials utilize bcryptjs salt rounds = 10 hashing. Passwords are never stored or logged in plain text anywhere in the database.

' JWT & Role-Based Authorization

Stateless JSON Web Tokens with 7-day expiration enforce backend authorization. Protected endpoints verify patient and doctor roles independently.

' Dual Transport Security

Supports secure HTTP-only cookies for Web clients and standard Bearer Authorization headers for Mobile clients.

' Zero Credential Exposure in Demonstration Package

Database connection strings (DATABASE_URL), database passwords, JWT secrets, and API keys are completely excluded from all demonstration files.

' 100% Synthetic Patient Data

All demonstration records (names, clinical complaints, phone numbers) are safe synthetic fixtures. No real patient data exists in this codebase.

' Backend Health Verification Endpoint

A dedicated endpoint GET /api/health/database safely checks PostgreSQL connectivity without leaking credentials.

Health Check API Response Sample (GET /api/health/database):


```
{"success": true, "configured": true, "connected": true, "mode": "postgresql", "database": "mediflow", "error": null}
```

7. Demonstration Dataset Summary

The package is populated with a balanced, realistic demonstration dataset from server/data/seedData.js representing a multi-city healthcare network.

5 Regional Cities

Chennai, Bangalore, Hyderabad, Coimbatore, and Madurai with mapped popular hospital areas.

70 Specialist Doctors

Balanced coverage across Cardiology, Dermatology, Orthopedics, Pediatrics, Neurology, ENT, and General Medicine.

5 Demo User Accounts

Pre-seeded demo credentials for patient (patient@mediflow.demo) and doctor (doctor@mediflow.demo).

16 Appointments

Active waiting room queues with tokens (Q-01, Q-02) and historical completed consultations.

2 Consultation Reports

Official structured medical discharge summaries with vitals, doctor observations, and advice.

3 Patient Notifications

Real-time alert records for queue turn calls, report downloads, and lab updates.

4 Family Member Profiles

Multi-generational family dependents with recorded chronic health histories.

4 Medical Vault Documents

Diagnostic blood test panels, 12-lead ECG reports, radiology PA views, and allergy panels.

Demonstration Package Ready:

Files created in MediFlow-Database/: README.txt, schema.sql, seed_data.sql, database-demo.pdf.
Suitable for academic presentation, architecture review, and live faculty demonstration.